1. Мои данные

Козенко Дмитрий; группа Б08; студенческий номер `st117373`.

2. Полное название задачи

Разбиение вершин дорожной сети между k государствами с заданными столицами методом локально жадного расширения по минимальной длине ребёрного пересечения.

3. Теоретическое описание задачи

Неориентированный взвешенный связный граф моделирует города и дороги. Задано k государств; у каждого государства указана вершина-столица. Требуется назначить каждой немаркированной вершине ровно одно государство так, чтобы к множество вершин каждого государства было связным с точностью до уже захваченного фронта процедуры расширения; на практике используется итеративная эвристика: поочерёдно каждое государство забирает ближайшую через ребро ещё неназначенную вершину. Дорога задаёт неотрицательный вес; исходную матрицу смежности хранят с отдельной «бесконечностью» для отсутствующих рёбер, но ключевой шаг роста смотрит только на простые дороги, инцидентные уже включённым в государство узлам.

4. Теоретическое описание решения

Парсинг. После фильтрации комментариев и пустых строк читаются n, m, блок из m рёбер $u\,v\,w$, число k и k номеров столиц.

Инициализация государств. Все столицы уникально помечаются своими индексами государства.

Жадный цикл захвата. Пока есть неназначенные вершины, цикл делает несколько параллельных фаз подряд для государств в порядке $1 \ldots k$: для множества вершин, уже принадлежащих государству s, отыскивается ребро минимальной длины к вершине, ещё не принадлежащей ни одному государству; такая вершина присоединяется к s. Если на полном сканировании набора государств ни одну вершину взять нельзя, разбиение недостаточно (граф между оставкой и уже помеченной частью содержит «разрыв»), сохраняется предупреждение.

Визуализация и экспорт. На консоль выводится табличное представление весов с цветными метками (при сборке отчёта ANSI убирается), опционально в файл сохраняется машиночитаемый текстовый дамп результатов группировки.

5. Программа — листинг `main.cpp`

Вспомогательные средства см. в каталоге `../common`. Ниже — `main.cpp` с подменой псевдографики Unicode на ASCII только для стабильной вёрстки PDF (точнее — экземпляр, сгенерированный тем же фильтром, что и блок вывода):

```
#include "../common/io_ru.hpp"
#include "../common/table_ru.hpp"

#include <cctype>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <tuple>
#include <vector>

namespace {

constexpr int NO_PATH = 1'000'000'000;

struct Road {
    int dst;
    int len;
};

std::string strip(std::string s) {
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.front()))) s.erase(0, 1);
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.back()))) s.pop_back();
    return s;
}

bool isCommentOrEmpty(const std::string& line) {
    const std::string t = strip(line);
    if (t.empty()) return true;
    if (t[0] == '#') return true;
    return false;
}

struct ParsedInput {
    int n = 0, m = 0, k = 0;
    std::vector<std::vector<Road>> g;
    std::vector<std::vector<int>> dist;
    std::vector<int> capitals;
    std::vector<std::tuple<int, int, int>> edges_ordered;  // как во входном файле
};

bool parseIntLine(std::string line, int& a, int& b) {
    line = strip(line);
    std::istringstream is(line);
    if (!(is >> a >> b)) return false;
    return true;
}

bool parseIntLine3(std::string line, int& a, int& b, int& c) {
    line = strip(line);
    std::istringstream is(line);
    if (!(is >> a >> b >> c)) return false;
    return true;
}

// Формат входа (текст):
//   Первая непустая не-комментария строка: n m
//   m строк: u v w
//   строка: k
//   строка: c1 c2 ... ck (k столиц)
// Строки с # в начале и пустые пропускаются.
bool loadFromStream(std::istream& in, ParsedInput& out, std::string& err) {
    std::vector<std::string> lines;
    std::string line;
    while (std::getline(in, line)) {
        if (!isCommentOrEmpty(line)) lines.push_back(strip(line));
    }
    if (lines.size() < 3) {
        err = "нужны строки: n m, рёбра, k, столицы";
        return false;
    }
    size_t p = 0;
```

```
    if (!parseIntLine(lines[p++], out.n, out.m)) {
        err = "ожидались n m";
        return false;
    }
    if (out.n < 1 || out.m < 0) {
        err = "n>=1, m>=0";
        return false;
    }
    if (lines.size() < static_cast<size_t>(1 + out.m + 2)) {
        err = "мало строк под рёбра и k";
        return false;
    }
    out.g.assign(out.n + 1, {});
    out.dist.assign(out.n + 1, std::vector<int>(out.n + 1, NO_PATH));
    for (int e = 0; e < out.m; ++e) {
        int a, b, w;
        if (!parseIntLine3(lines[p++], a, b, w)) {
            err = "ожидалось ребро u v w";
            return false;
        }
        if (a < 1 || a > out.n || b < 1 || b > out.n) {
            err = "индексы вершин 1..n";
            return false;
        }
        out.g[a].push_back({b, w});
        out.g[b].push_back({a, w});
        out.dist[a][b] = out.dist[b][a] = w;
        out.edges_ordered.push_back({a, b, w});
    }
    for (int v = 1; v <= out.n; ++v) out.dist[v][v] = 0;

    {
        std::istringstream ks(lines[p++]);
        if (!(ks >> out.k) || out.k < 1) {
            err = "ожидалось k (число государств)";
            return false;
        }
    }
    {
        std::istringstream is(lines[p++]);
        out.capitals.resize(out.k);
        for (int i = 0; i < out.k; ++i) {
            if (!(is >> out.capitals[i])) {
                err = "ожидалось k столиц";
                return false;
            }
        }
    }
    for (int c : out.capitals) {
        if (c < 1 || c > out.n) {
            err = "столица вне 1..n";
            return false;
        }
    }
    return true;
}

int pickNextCity(const std::vector<int>& byState, const std::vector<std::vector<Road>>& g,
                 const std::vector<int>& belongs) {
    int best = -1;
    int bestLen = NO_PATH;
    for (int u : byState) {
        for (const Road& r : g[u]) {
            if (belongs[r.dst] != 0) continue;
            if (r.len < bestLen) {
                bestLen = r.len;
                best = r.dst;
            }
        }
    }
    return best;
}
```

```
void runPartition(int n, int k, const std::vector<std::vector<Road>>& g,
                  const std::vector<int>& capitals, std::vector<int>& belongs,
                  std::vector<std::vector<int>>& perState, std::string& warn) {
    belongs.assign(n + 1, 0);
    perState.assign(k + 1, {});
    int done = 0;
    for (int s = 1; s <= k; ++s) {
        int cap = capitals[s - 1];
        belongs[cap] = s;
        perState[s].push_back(cap);
        ++done;
    }
    for (; done < n;) {
        bool progress = false;
        for (int s = 1; s <= k && done < n; ++s) {
            int t = pickNextCity(perState[s], g, belongs);
            if (t < 0) continue;
            belongs[t] = s;
            perState[s].push_back(t);
            ++done;
            progress = true;
        }
        if (!progress) {
            warn = "нельзя назначить все города (разрыв графа или нет путей).";
            break;
        }
    }
}

void writeResultText(std::ostream& os, int n, int k, const std::vector<int>& belongs,
                     const std::vector<std::vector<int>>& perState, const std::string& warn) {
    os << "# результат группировки (КА-разбиение по рёбрам)\n";
    os << "n " << n << "\n";
    os << "k " << k << "\n";
    os << "belongs\n";
    for (int v = 1; v <= n; ++v) os << v << " " << belongs[v] << "\n";
    os << "groups\n";
    for (int s = 1; s <= k; ++s) {
        os << s;
        for (int c : perState[s]) os << " " << c;
        os << "\n";
    }
    if (!warn.empty()) os << "warning " << warn << "\n";
}

std::string ansiState(int id) {
    if (id == 0) return "\033[0m";
    int code = 30 + (id % 6 == 0 ? 6 : id % 6);
    return "\033[" + std::to_string(code) + "m";
}

void printPseudographRu(std::ostream& out, int n, int k, const std::vector<int>& belongs,
                        const std::vector<std::vector<int>>& perState,
                        const std::vector<std::vector<int>>& dist, const std::string& warn,
                        bool useAnsi) {
    auto a = [useAnsi](int id) { return useAnsi ? ansiState(id) : std::string(); };
    const std::string reset = useAnsi ? "\033[0m" : "";

    out << "\n";

    std::vector<std::string> th = {"Город", "Гос-", "Примечание"};
    std::vector<std::vector<std::string>> city_rows;
    city_rows.reserve(static_cast<size_t>(n));
    for (int v = 1; v <= n; ++v) {
        int s = belongs[v];
        std::string note;
        for (int t = 1; t <= k; ++t) {
            if (perState[t].empty()) continue;
            if (perState[t][0] == v) {
                note = "столица ";
                note += std::to_string(s);
                break;
            }
```

```
        }
        if (note.empty()) note = "-";
        city_rows.push_back({std::to_string(v), std::to_string(s), note});
    }
    table_ru::print_table(out, {}, th, city_rows);

    out << "\n  Матрица весов (строка/колонка: цвет = государство)\n\n";
    out << "      ";
    for (int j = 1; j <= n; ++j) {
        out << a(belongs[j]) << std::setw(4) << j << (useAnsi ? ansiState(0) : "");
    }
    out << "\n      ";
    for (int j = 1; j <= n; ++j) out << "----";
    out << '\n';
    for (int i = 1; i <= n; ++i) {
        out << a(belongs[i]) << std::setw(2) << i << " | " << (useAnsi ? ansiState(0) : "");
        for (int j = 1; j <= n; ++j) {
            if (dist[i][j] == NO_PATH)
                out << std::setw(4) << '-';
            else
                out << std::setw(4) << dist[i][j];
        }
        out << '\n';
    }
    out << "\n  Состав государств\n\n";
    for (int s = 1; s <= k; ++s) {
        out << "  [" << s << "] ";
        for (size_t i = 0; i < perState[s].size(); ++i) {
            if (i) out << " · ";
            out << perState[s][i];
        }
        out << "\n";
    }
    out << "\n";
}

void print_loaded_input_tables(std::ostream& os, const ParsedInput& in) {
    std::vector<std::vector<std::string>> edge_rows;
    edge_rows.reserve(in.edges_ordered.size());
    for (size_t i = 0; i < in.edges_ordered.size(); ++i) {
        const auto& [u, v, w] = in.edges_ordered[i];
        edge_rows.push_back({std::to_string(i + 1), std::to_string(u), std::to_string(v),
                             std::to_string(w)});
    }
    table_ru::print_table(os, "Входные данные: рёбра (порядок как в файле)",
                          {"Номер", "u", "v", "Вес"}, edge_rows);

    // table_ru::print_table(os, "Сводка параметров графа", {"Параметр", "Значение"},
    //                       {
    //                           {"Число вершин n", std::to_string(in.n)},
    //                           {"Число рёбер m", std::to_string(in.m)},
    //                           {"Число государств k", std::to_string(in.k)},
    //                       });

    std::vector<std::vector<std::string>> cap_rows;
    for (size_t i = 0; i < in.capitals.size(); ++i)
        cap_rows.push_back({std::to_string(i + 1), std::to_string(in.capitals[i])});
    table_ru::print_table(os, "Столицы государств", {"Государство №", "Вершина-столица"},
                          cap_rows);
}

}  // namespace

int main(int argc, char** argv) {
    std::ifstream fin;
    std::ofstream fout;
    if (!io_ru::open_read_optional_write(argc, argv, fin, &fout, std::cerr,
                                         "<вход.txt> [выход.txt - опционально]")) {
        return 2;
    }

    ParsedInput in;
    std::string err;
```

```
    if (!loadFromStream(fin, in, err)) {
        std::cerr << "Файл «" << argv[1] << "» содержит ошибку формата: " << err << '\n';
        return 1;
    }

    std::ostream* outFilePtr =
        fout.is_open() ? static_cast<std::ostream*>(&fout) : nullptr;

    print_loaded_input_tables(std::cout, in);

    std::vector<int> belongs;
    std::vector<std::vector<int>> perState;
    std::string warn;
    runPartition(in.n, in.k, in.g, in.capitals, belongs, perState, warn);

    printPseudographRu(std::cout, in.n, in.k, belongs, perState, in.dist, warn, true);
    if (outFilePtr) {
        print_loaded_input_tables(*outFilePtr, in);
        printPseudographRu(*outFilePtr, in.n, in.k, belongs, perState, in.dist, warn, false);
        writeResultText(*outFilePtr, in.n, in.k, belongs, perState, warn);
    }
    return 0;
}
```

6. Комментарии к коду

`loadFromStream` — строчный сборник блока строк и последовательный разбор. `pickNextCity` — поиск смежной неназначенной вершины с минимальным ценниковым ребром из текущего фронта государства (через `Road.dst` и длину). `runPartition` — глобальная фазная жадность с флагами прогресса и возможностью ошибки блокировки. `printPseudographRu` — матрица весов плюс раскладка подписей.

7. Разбор входного файла `sample.txt`

```
# n m, затем m рёбер u v w, затем k, затем k столиц
5 6
1 2 1
1 3 2
2 4 1
3 4 3
3 5 1
4 5 2
3
1 3 5
```

Граф: n=5, m=6: цепочка 1−2−4−5, при этом от 1 ведёт ветка в 3, а 3 связывается также с 4 и с 5 (см. числовые строки входа).

Столицы: государство 1 имеет столицу 1; государство 2 — вершину 3; государство 3 — вершину 5. Жадный алгоритм поочереди расширяет территории через самые короткие примыкающие рёбра; на связном этом примере все свободные вершины (в частности узел 2 между столицами) будут впоследствии делены между блоками через конкретные локальные выборы `pickNextCity`, что можно проследить по таблице рёбер в выводе и по итоговой колонке «Государство».

8. Пример вывода программы при запуске на `sample.txt`

Ниже — вывод программы после фильтрации ANSI-последовательностей и псевдографики таблицы (для включения в `pdflatex`); пояснения см. после листинга.

```
==============================================================
  Входные данные: рёбра (порядок как в файле)
==============================================================
+-------+----+----+-----+
| Номер | u  | v  | Вес |
+-------+----+----+-----+
|     1 |  1 |  2 |   1 |
|     2 |  1 |  3 |   2 |
|     3 |  2 |  4 |   1 |
|     4 |  3 |  4 |   3 |
|     5 |  3 |  5 |   1 |
|     6 |  4 |  5 |   2 |
+-------+----+----+-----+

==============================================================
  Столицы государств
==============================================================
+---------------+-----------------+
| Государство № | Вершина-столица |
+---------------+-----------------+
|             1 |               1 |
|             2 |               3 |
|             3 |               5 |
+---------------+-----------------+

+-------+------+------------+
| Город | Гос- | Примечание |
+-------+------+------------+
|     1 |    1 |  столица 1 |
|     2 |    1 |          - |
|     3 |    2 |  столица 2 |
|     4 |    2 |          - |
|     5 |    3 |  столица 3 |
+-------+------+------------+

  Матрица весов (строка/колонка: цвет = государство)

         1   2   3   4   5
      --------------------
 1 |    0   1   2   -   -
 2 |    1   0   -   1   -
 3 |    2   -   0   3   1
 4 |    -   1   3   0   2
 5 |    -   -   1   2   0

  Состав государств

  [1] 1 · 2
  [2] 3 · 4
  [3] 5
```

Комментарий. Первая таблица дублирует входные дороги. Табличный блок городов содержит присвоенный государствам индекс; для столиц столбец «Примечание» отмечает специально.

Матрица весов копирует исходные длины на рёбрах с «бесконечностью» вне смежности. Блок «Состав государств» упорядоченно расписывает, какие вершины заняло каждое государство после фаз жадности.

9. Ссылка на исходный код

Исходный код задачи доступен по ссылке:
`https://github.com/mitya-y/formal-theories-tp-3-course/tree/master/Столицы`